

Pet Paradise — Full Project Explanation

What is this project?

Pet Paradise is a React-based Single Page Application (SPA) where users can:

- Browse pets (Dogs, Cats, Birds, Fish, Small Pets)
- Click a pet to see its details
- Add pets to a cart
- Fill an enquiry form
- Cart data is saved even after refreshing the page

The project is built to cover all 8 topics in the rubric.

File Structure (What each file does)

```
src/  
  context/CartContext.jsx    → Global cart state (Context API)  
  components/  
    ErrorBoundary.jsx       → Catches errors, shows fallback UI  
    PetCard.jsx             → Displays one pet card (dumb component)  
  pages/  
    Home.jsx                → Main page with filter + search (smart  
component)  
    PetDetail.jsx           → Single pet detail page (/pet/:id)  
    Cart.jsx                → Cart page  
    Enquiry.jsx             → Enquiry form with validation  
  data/pets.js              → All pet data (array of objects)  
  App.jsx                   → Routing + Header + Global styles  
  main.jsx                  → Entry point (renders App into index.html)  
  tests/  
    PetCard.test.jsx        → 2 unit tests  
    Cart.test.jsx           → 1 integration test  
    setup.js                → Test configuration
```

TOPIC 1 — Controlled & Uncontrolled Components (10 marks)

File: src/pages/Enquiry.jsx

What is a Controlled Component?

A controlled component is an input whose value is controlled by React state (useState). Every keystroke updates the state, and the input always shows the state value.

```
jsx

const [phone, setPhone] = useState("");

<input
  value={phone} // React controls the value
  onChange={(e) => setPhone(e.target.value)} // state updates on every key
/>
```

Here, the phone number input is controlled. React always knows what is typed.

What is an Uncontrolled Component?

An uncontrolled component manages its own value internally in the DOM. We access the value only when needed (like on form submit) using useRef.

```
jsx

const nameRef = useRef(null);
const messageRef = useRef(null);

<input ref={nameRef} type="text" /> // DOM manages this
<textarea ref={messageRef} /> // DOM manages this
```

We read the value with nameRef.current.value only when the form is submitted.

Form Validation

```
jsx
```

```
function validate() {
  const errs = {};
  if (!nameRef.current?.value.trim()) errs.name = "Name is required.";
  if (!/^[6-9]\d{9}$/.test(phone)) errs.phone = "Enter valid 10-digit number.";
  if (!messageRef.current?.value.trim()) errs.message = "Message is required.";
  return errs;
}
```

- Name and Message → validated via useRef (uncontrolled)
- Phone → validated via useState (controlled)
- The regex `^[6-9]\d{9}$` checks for valid Indian mobile numbers

TOPIC 2 — Container / Presenter Components (20 marks)

Container (Smart) Component: `src/pages/Home.jsx`

A container component contains business logic, state management, and data fetching. It does NOT worry about how things look.

jsx

```

function Home() {
  const [activeCategory, setActiveCategory] = useState("All"); // owns state
  const [search, setSearch] = useState(""); // owns state
  const { addToCart } = useCart(); // uses context
  const navigate = useNavigate(); // handles navigation

  const filtered = pets.filter(...); // business logic - filtering

  return (
    <div>
      {filtered.map((pet) => (
        <PetCard
          pet={pet}
          onAdd={addToCart} // passes logic down as props
          onView={(id) => navigate(`/pet/${id}`)}
        />
      ))}
    </div>
  );
}

```

Home is smart — it decides WHAT data to show and WHAT happens on user actions.

Presenter (Dumb) Component: src/components/PetCard.jsx

A presenter component only receives props and renders UI. No state, no logic.

```

jsx

function PetCard({ pet, onAdd, onView }) { // only props - no useState
  return (
    <div className="pet-card">
      <img src={pet.image} onClick={() => onView(pet.id)} />
      <h3>{pet.name}</h3>
      <span>₹{pet.price.toLocaleString("en-IN")}</span>
      <button onClick={() => onAdd(pet)}>Add to Cart</button>
    </div>
  );
}

```

PetCard is dumb — it just renders whatever data it receives. It has no idea where the data comes from or what happens after clicking. That is handled by Home.

TOPIC 3 — Error Boundary & Fallback Component (10 marks)

File: src/components/ErrorBoundary.jsx

Error Boundaries only work as class components in React. They catch JavaScript errors anywhere in their child component tree.

```
jsx

class ErrorBoundary extends Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  // Called when any child throws an error
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  // Used for logging the error
  componentDidCatch(error, info) {
    console.error("ErrorBoundary caught:", error, info);
  }

  render() {
    if (this.state.hasError) {
      return <FallbackUI message={this.state.error?.message} />;
    }
    return this.props.children; // render normally if no error
  }
}
```

Fallback Component

```
jsx
```

```
function FallbackUI({ message, onReset }) {
  return (
    <div>
      <h2>Something went wrong</h2>
      <p>{message}</p>
      <button onClick={onReset}>Try Again</button>
    </div>
  );
}
```

FallbackUI is shown instead of a blank/crashed screen when an error occurs.

Where is it used? (App.jsx)

```
jsx

<ErrorBoundary>
  <Header />
  <Routes>...</Routes>
</ErrorBoundary>
```

It wraps the entire app so any crash shows a friendly message instead of white screen.

TOPIC 4 — Browser Storage with useEffect (10 marks)

File: src/context/CartContext.jsx

Reading from localStorage on first load (lazy initializer)

```
jsx

const [cart, setCart] = useState(() => {
  try {
    const saved = localStorage.getItem("pp_cart");
    return saved ? JSON.parse(saved) : [];
  } catch {
    return [];
  }
});
```

When the app loads for the first time, it checks if there is saved cart data in localStorage. If yes, it restores it. This is called a lazy initializer.

Saving to localStorage whenever cart changes (useEffect)

```
jsx

useEffect(() => {
  localStorage.setItem("pp_cart", JSON.stringify(cart));
}, [cart]); // runs every time "cart" state changes
```

The dependency array [cart] means: "run this effect whenever cart changes." So every time user adds or removes a pet, the cart is saved to localStorage.

Why localStorage and not sessionStorage?

- localStorage persists even after closing the browser tab
 - sessionStorage clears when the tab is closed
 - For a cart, we want data to stay, so localStorage is the right choice
-

TOPIC 5 — State Management with Context API (20 marks)

File: src/context/CartContext.jsx

Why Context API?

Without Context, we would need to pass cart data as props from App → Home → PetCard. This is called "prop drilling" and becomes messy. Context makes state global.

Creating Context

```
jsx

const CartContext = createContext();
```

Providing State (CartProvider)

```
jsx
```

```

export function CartProvider({ children }) {
  const [cart, setCart] = useState([]);

  // Global state
  // Derived state - calculated from cart, not stored separately
  const totalItems = cart.reduce((sum, item) => sum + item.qty, 0);
  const totalPrice = cart.reduce((sum, item) => sum + item.price * item.qty, 0)

  return (
    <CartContext.Provider value={{ cart, totalItems, totalPrice, addToCart, removeItem }}>
      {children}
    </CartContext.Provider>
  );
}

```

- Global state: cart array
- Derived state: totalItems and totalPrice are computed from cart, not stored separately
- State colocation: all cart-related state lives together in CartContext

Consuming Context (any component)

```

jsx

export function useCart() {
  return useContext(CartContext);
}

// Used in any component like:
const { cart, addToCart, totalItems } = useCart();

```

Where is CartProvider used? (App.jsx)

```

jsx

<CartProvider>
  <Header /> // can use useCart()
  <Routes>
    <Route path="/" element={<Home />} /> // can use useCart()
    <Route path="/cart" element={<Cart />} /> // can use useCart()
  </Routes>
</CartProvider>

```


Any component inside CartProvider can access cart state.

TOPIC 6 — SPA Routing (10 marks)

File: src/App.jsx

What is SPA Routing?

In a normal website, clicking a link loads a new HTML page from the server. In a SPA (Single Page Application), React Router changes the URL and renders a different component — without any page reload.

Route Types in this project:

1. Basic Route

```
jsx

<Route path="/" element={<Home />} />
<Route path="/cart" element={<Cart />} />
```

Simple fixed URL → component mapping.

2. Dynamic Route

```
jsx

<Route path="/pet/:id" element={<PetDetail />} />
```

The `(:id)` is a URL parameter. `/pet/1`, `/pet/2`, `/pet/5` all work. In `PetDetail.jsx`, we read it:

```
jsx

const { id } = useParams();
const pet = pets.find((p) => p.id === Number(id));
```

3. Protected Route

```
jsx
```

```
function ProtectedEnquiry() {
  const { totalItems } = useCart();
  return totalItems > 0 ? <Enquiry /> : <Navigate to="/cart" replace />;
}

<Route path="/enquiry" element={<ProtectedEnquiry />} />
```

If user tries to go to /enquiry without adding anything to cart, they are redirected to /cart automatically.

4. Fallback Route

```
jsx

<Route path="*" element={<Navigate to="/" replace />} />
```

Any unknown URL redirects back to Home.

TOPIC 7 — Unit & Integration Testing (10 marks)

File: src/tests/PetCard.test.jsx — Unit Tests

Unit tests test one component in isolation.

Unit Test 1 — Renders correctly

```
jsx

it("renders pet name and price correctly", () => {
  render(<PetCard pet={mockPet} onAdd={vi.fn()} onView={vi.fn()} />);
  expect(screen.getByText("Golden Retriever")).toBeInTheDocument();
  expect(screen.getByText(/25,000/)).toBeInTheDocument();
});
```

Checks that PetCard shows the pet name and price.

Unit Test 2 — Button calls correct function

```
jsx
```

```
it("calls onAdd with pet when Add to Cart is clicked", () => {
  const onAdd = vi.fn(); // mock function
  render(<PetCard pet={mockPet} onAdd={onAdd} onView={vi.fn()} />);
  fireEvent.click(screen.getByTestId("add-btn"));
  expect(onAdd).toHaveBeenCalledWith(mockPet);
});
```

Checks that clicking "Add to Cart" button calls onAdd with the correct pet.

File: src/tests/Cart.test.jsx — Integration Test

Integration tests test multiple components working together.

```
jsx

it("shows empty cart message when no items added", () => {
  render(<Cart />, { wrapper: Wrapper }); // Cart + CartProvider + Router together
  expect(screen.getByText(/your cart is empty/i)).toBeInTheDocument();
});
```

Tests that Cart component works correctly with CartContext together.

Test Results: 4/4 tests passed

TOPIC 8 — Deployment

Steps to deploy:

GitHub

```
bash

git init
git add .
git commit -m "Pet Paradise project"
git remote add origin https://github.com/yourusername/petparadise.git
git push -u origin main
```

Netlify / Vercel

1. Go to netlify.com or vercel.com
2. Click "New Project" → Import from GitHub

3. Build command: `npm run build`

4. Publish directory: `dist`

5. Click Deploy

EXPECTED FACULTY QUESTIONS & ANSWERS

Q1: What is the difference between controlled and uncontrolled components?

A: In a controlled component, the input's value is stored in React state using `useState`. Every change updates the state and React re-renders. Example: our phone field. In an uncontrolled component, the DOM manages the input value. We use `useRef` to read the value only when needed, like on form submit. Example: our name and message fields.

Q2: Why did you use `useRef` instead of `useState` for the name field?

A: For the name field, we don't need to validate or react to every keystroke. We only need the value when the form is submitted. `useRef` is more efficient here because it doesn't cause a re-render on every keystroke, unlike `useState`.

Q3: What is the difference between a container and a presenter component?

A: A container (smart) component contains logic — it manages state, fetches data, and handles events. `Home.jsx` is our container — it filters pets, manages search state. A presenter (dumb) component only receives props and renders UI. It has no logic. `PetCard.jsx` is our presenter — it just displays whatever data it receives.

Q4: Why do we need an Error Boundary? Can we use a function component?

A: Error Boundaries catch JavaScript errors during rendering and show a fallback UI instead of a white crashed screen. We cannot use function components for Error Boundaries because they require two lifecycle methods: `getDerivedStateFromError` and `componentDidCatch`, which only exist in class components. That is why `ErrorBoundary.jsx` is a class component.

Q5: What is the difference between `localStorage` and `sessionStorage`?

A: localStorage persists data permanently until manually cleared. sessionStorage data is lost when the browser tab is closed. We use localStorage for the cart so that if the user refreshes the page or comes back later, their cart is still there.

Q6: What is derived state? Where did you use it?

A: Derived state is state that is calculated from existing state rather than stored separately. In CartContext, totalItems and totalPrice are derived state:

```
jsx
const totalItems = cart.reduce((sum, item) => sum + item.qty, 0);
const totalPrice = cart.reduce((sum, item) => sum + item.price * item.qty, 0);
```

We do not store totalItems or totalPrice separately. They are always calculated from cart. This avoids data going out of sync.

Q7: What is prop drilling and how does Context API solve it?

A: Prop drilling is when you pass data through multiple components as props just to reach a deeply nested component. For example: App → Home → PetCard → button, passing cart data at every level. Context API creates a global store. Any component can access cart data directly using useCart() without passing props through every parent.

Q8: What is a protected route? How did you implement it?

A: A protected route only allows access if a condition is met. In our project, /enquiry is protected — you can only visit it if your cart has at least one item.

```
jsx
function ProtectedEnquiry() {
  const { totalItems } = useCart();
  return totalItems > 0 ? <Enquiry /> : <Navigate to="/cart" replace />;
}
```

If cart is empty, user is redirected to /cart automatically.

Q9: What is a dynamic route? Where did you use it?

A: A dynamic route has a URL parameter that changes per page. We used `/pet/:id` — the `:id` part is dynamic. `/pet/1` shows Golden Retriever, `/pet/4` shows Persian Cat. In `PetDetail.jsx`, we use `useParams()` to read the `id` from the URL and find the matching pet from our data.

Q10: What is the difference between unit testing and integration testing?

A: A unit test tests one component in isolation. We test `PetCard` alone — does it render the name? Does the button work? We mock all external dependencies. An integration test tests multiple components working together. We test `Cart` with `CartContext` together to verify they integrate correctly.

Q11: What is `useEffect` and what is the dependency array?

A: `useEffect` runs side effects after render. The dependency array controls when it runs.

```
jsx
useEffect(() => {
  localStorage.setItem("pp_cart", JSON.stringify(cart));
}, [cart]);
```

The `[cart]` dependency means: run this effect only when `cart` changes. An empty `[]` means run only once when component mounts. No array means run after every render.

Q12: What is state colocation?

A: State colocation means keeping state as close as possible to where it is used. `Cart` state is only needed by `cart`-related components, so it lives in `CartContext` — not in `App.jsx` or `main.jsx`. If only one component needs state, keep it in that component. If multiple components need it, lift it to their common parent or context.

Q13: Why does the cart persist after page refresh?

A: Because we save the `cart` to `localStorage` every time it changes using `useEffect`. When the app loads again, `useState` reads from `localStorage` using a lazy initializer:

```
jsx
```

```
const [cart, setCart] = useState(() => {
  const saved = localStorage.getItem("pp_cart");
  return saved ? JSON.parse(saved) : [];
});
```

So the cart is restored from localStorage on startup.

Q14: What is the purpose of the lazy initializer in useState?

A: Normally, useState runs its initial value on every render. If we wrote `useState(JSON.parse(localStorage.getItem("pp_cart")))`, it would call `JSON.parse` on every render, which is wasteful. By passing a function `() => {...}`, React only calls it once on the first render. This is called a lazy initializer and improves performance.

Q15: How does your app work without page reloads?

A: We use React Router (react-router-dom). `BrowserRouter` wraps the app. `Routes` component defines which component to show for each URL. `Link` and `Navigate` components change the URL without reloading. React intercepts the browser's navigation and renders the matching component instead.

Quick Summary Table

Topic	File	Key Concept
useState + useRef + validation	Enquiry.jsx	Controlled vs uncontrolled inputs
Container / Presenter	Home.jsx + PetCard.jsx	Smart vs dumb components
Error Boundary	ErrorBoundary.jsx	Class component, <code>getDerivedStateFromError</code>
localStorage + useEffect	CartContext.jsx	Persist state to browser storage
Context API	CartContext.jsx	Global state, derived state, colocation
Routing	App.jsx	Basic, dynamic (<code>/pet/:id</code>), protected

Topic	File	Key Concept
Testing	tests/	2 unit + 1 integration, vitest
Deployment	—	npm run build → dist → Netlify/Vercel